

Akhil Sathish Kumar

Presenter Pack

Member B — Maze Generation & Difficulty Balance

Slides 1 & 5

Maze Runner · Problem Statement 15

REVA University · Hackathon अभिनव · 07/10/2026

SECTION 1 — The project (everyone must know this)

What we built

A maze game where the player has to reach the exit while an enemy chases them using real pathfinding. The enemy recalculates the shortest route to the player on every single step, using either BFS or A*, and you can switch between the two live and watch the difference on screen.

From level 4 a **second enemy** joins running the *other* algorithm, so BFS and A* hunt the same player at the same time.

How it works, in 60 seconds

The maze is a grid of 1s and 0s — 1 is a wall, 0 is floor. The player and each enemy sit on one cell. Sixty times a second we read the keyboard, move everything one cell if its cooldown has expired, and redraw. The enemy decides where to go by running a search from itself to the player, then stepping onto the second cell of the route it found.

Everything else is detail on top of that.

The five facts everyone must be able to say

1. **BFS uses a FIFO queue, so it finds the shortest path.** It finishes every cell at distance d before touching anything at $d+1$, so the first time it reaches the goal it got there in the fewest steps.
2. **A* scores cells** $f = g + h$ — g is steps taken, h is Manhattan distance remaining. h must never overestimate or optimality is lost.
3. **A* expanded ~26% fewer cells but ran slower** in wall-clock time, because its heap costs $O(\log n)$ per operation against the deque's $O(1)$.
4. **The enemy used to camp the exit** — "farthest from the player" is the opposite corner, which is where the exit is. Fixed by maximising `min(distance to player, distance to exit)`.
5. **We tuned difficulty by measurement, not feel** — `autoplay.py` ran 500 bot playthroughs and found the enemy was only $1.1\times$ slower than the player.

The numbers

Tech	Python 3 + pygame-ce. No dataset, no network, no hardware
Tests	226 headless checks, all passing
Benchmark	50 mazes per configuration
Balance	500 bot playthroughs; L1–3 ~68–70%, L4–10 20–38%
Repo	github.com/Hyperval/maze-runner

Who built what

Person	Owns
Akhil Sathish Kumar	Maze generation, braiding, difficulty balance
Akshay N	Pathfinding — BFS, A*, distance maps
Abdullah Subhaan K	Rendering, menu, pause, HUD, test suite
Christen Mendis	benchmark.py, autoplay.py, docs, demo video

SECTION 2 — Your code: `maze.py` and the balance tuning

You own how the maze is built and how hard the game is. Two files: `maze.py`, and the tuning constants in `settings.py`.

How the maze is generated — recursive backtracker

1. Start at cell (1,1), mark it floor.
2. Look at neighbours TWO cells away that are still solid wall.
3. Pick one at random. Knock out the wall BETWEEN them. Move there.
4. No unvisited neighbours? Pop the stack and back up.
5. Done when the stack is empty.

It is depth-first search used to **carve** rather than to search.

The two questions you will be asked

"Why two cells away?"

So walls always land on even coordinates and stay one cell thick. If we carved one cell at a time the whole grid would dissolve into one open room with no walls at all.

"Why must COLS and ROWS be odd?"

The carver only ever stands on odd coordinates. An even width would leave a ragged half-carved column at the edge.

Braiding — your most interesting decision

A freshly carved maze is "**perfect**": exactly one route between any two cells. That sounds good but means it is nothing but dead ends — the player gets cornered with zero counterplay.

`braid()` finds dead ends (cells with exactly one open side) and knocks out one extra wall with 35% probability, creating loops and therefore escape routes.

It had a second effect we measured. Loops also widen A*'s advantage over BFS, from ~11% to ~26%, because the search finally has real alternatives to discriminate between. A tight maze constrains the corridors so much that the heuristic has nothing to choose between.

That is a genuinely good thing to say out loud — it shows a design decision that turned out to have a measurable second consequence you didn't plan for.

`carve_steps()` and the `yield` keyword

```
def carve_steps(...):  
    ...  
    yield grid          # hand the grid back, then PAUSE here
```

A function containing `yield` is a **generator**. It doesn't run to completion — each `yield` hands a value to whoever is looping over it and freezes there until the next iteration asks for more. That's what lets us draw the maze being carved one step at a time without storing hundreds of copies.

The balance constants you tuned

Constant	Value	Effect
<code>PLAYER_MOVE_DELAY</code>	90 ms	How often the player can step
<code>ENEMY_MOVE_DELAY</code>	190 ms	Starting enemy speed
<code>ENEMY_SPEEDUP_PER_LEVEL</code>	11 ms	How fast difficulty ramps
<code>ENEMY_MIN_DELAY</code>	140 ms	Floor — the fastest the enemy ever gets
<code>ENEMY_HEAD_START_MS</code>	2200 ms	Enemies hold still at level start
<code>BRAID_CHANCE</code>	0.35	Fraction of dead ends opened into loops

The single most important relationship: `ENEMY_MIN_DELAY` (140) must stay **above** `PLAYER_MOVE_DELAY` (90). If the enemy's delay drops below the player's, the enemy moves more often than you do, and since it always walks a shortest path it catches you with mathematical certainty. Skill stops mattering.

We originally had it at 100ms — only 1.1× slower than the player — and the bot tester found the late levels were effectively unwinnable.

Your likely viva questions

- "How is the maze generated?" — recursive backtracker, above.
- "What is braiding and why?" — perfect maze = all dead ends = no counterplay.
- "How did you pick the difficulty numbers?" — measured with `autoplay.py`, tuned until L1–3 sat around 70% and later levels around 20–40%.
- "What happens if the enemy is faster than the player?" — it catches you with certainty, which is why the floor is above the player's delay.

SECTION 3 — Your presenting job

You present **Slide 1 (Cover)** and **Slide 5 (Future Scope)**. You open the presentation and you close it. Keep both short — the middle three slides and the live demo are where the marks are.

Slide 1 — Cover (target: 20 seconds)

Say roughly this:

Good morning. We're presenting problem statement 15 — a maze game where the enemy chases the player using pathfinding. I'm Akhil, and this is Akshay, Abdullah and Christen.

Then hand to Akshay. **Do not** read out the three cards on the slide; the examiners can see them. The temptation is to explain everything on the cover — don't, you'll run out of time for the demo.

If they ask what you personally built, one line: *"I built the maze generation and tuned the difficulty."*

Slide 5 — Future Scope (target: 40 seconds)

Pick ONE item and talk about it properly, rather than reading all four.

Best choice is the **predictive enemy**, because it names a real limitation of what we built:

Right now the enemy always paths to where the player *is*, so it trails you. With more time we'd have it path to where you're *heading*, so it could cut you off at a junction instead of following. That's a different problem — you have to predict intent, not just compute a route.

Then close:

That's our project. The code and documentation are on GitHub. Happy to take questions.

Don't trail off. Finish on a clear sentence and stop talking.

During the rest of the presentation

- **You answer anything about the maze or difficulty**, even on someone else's slide. If an examiner asks Christen "why is level 1 easier than level 5", that's yours — step in.
- You also own the question *"is the game actually winnable?"* — the answer is the 500-run measurement, L1–3 around 68–70%.

If something goes wrong

- Demo crashes → Christen plays the backup video. Don't apologise repeatedly; one sentence, move on.
- You blank on a question → *"Akshay owns that part, he can give you the detail."* Handing over cleanly is fine. Silence is not.

Final checklist

- [] Read Section 1 of this document until the five facts are automatic
- [] Practise your 20-second cover opener out loud, twice
- [] Know which ONE future-scope item you'll talk about
- [] Have `maze.py` open in a tab in case they ask to see the carving code